

Reporte de Vulnerabilidad

Archivo: CustomerController.cs

Código Analizado:

```
?using Castle.Core.Resource;
using FermaOrders.API.Controllers.Response;
using FermaOrders.Application.Dto.Components.Customer;
using FermaOrders.Application.Interface.Components;
using FermaOrders.Application.Service.Components;
using FermaOrders.Domain.Entities;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using System.Data;
using System.Net;

namespace FermaOrders.API.Controllers.Application
{
    [Route("api/[controller]")]
    [ApiController]
    public class CustomerController : ControllerBase
    {
        private readonly ICustomerService _customerService;

        public CustomerController(ICustomerService customerService)
        {
            _customerService = customerService;
        }

        // Crear un asiento
        [Authorize(Roles = "Admin")]
        [HttpPost]
        [ProducesResponseType(StatusCodes.Status200OK)]
        [ProducesResponseType(StatusCodes.Status403Forbidden)]
        [ProducesResponseType(StatusCodes.Status500InternalServerError)]
        public async Task<IActionResult> CreateCustomerAsync([FromBody] CustomerCreatedDto
customerDto)
        {
            var response = new RespuestaAPI();
            try
            {
                if (customerDto == null)
                {
                    return BadRequest("Invalid customer data.");
                }

                var customer = await _customerService.CreateCustomerAsync(customerDto);
                response.Result = customer;
                response.IsSuccess = true;
                response.StatusCode = HttpStatusCode.OK;
                return Ok(response);
            }
            catch (Exception ex)
            {
                response.StatusCode = HttpStatusCode.InternalServerError;
                response.IsSuccess = false;
                response.ErrorMessages.Add($"Error: {ex.Message}");
                return StatusCode(500, response);
            }
        }

        // Obtener un asiento por ID
        [Authorize(Roles = "Admin")]
        [HttpGet("{id}")]
        [ProducesResponseType(StatusCodes.Status200OK)]
        [ProducesResponseType(StatusCodes.Status403Forbidden)]
        [ProducesResponseType(StatusCodes.Status500InternalServerError)]
        public async Task<IActionResult> GetCustomerByIdAsync(int id)
        {
            var response = new RespuestaAPI();

            try
            {
```

```

        var customer = await _customerService.GetCustomerIdAsync(id);
        if (customer == null)
        {
            return BadRequest("Invalid customer data.");
        }

        response.Result = customer;
        response.IsSuccess = true;
        response.StatusCode = HttpStatusCode.OK;
        return Ok(response);
    }
    catch (Exception ex)
    {
        response.StatusCode = HttpStatusCode.InternalServerError;
        response.IsSuccess = false;
        response.ErrorMessages.Add($"Error: {ex.Message}");
        return StatusCode(500, response);
    }
}

[Authorize(Roles = "Admin")]
[HttpPut("{id}")]
[ProducesResponseType(StatusCodes.Status200OK)]
[ProducesResponseType(StatusCodes.Status400BadRequest)]
[ProducesResponseType(StatusCodes.Status500InternalServerError)]
public async Task<ActionResult> UpdateCustomerAsync(int id, [FromBody]
CustomerUpdateDto customerDto)
{
    var response = new RespuestaAPI();

    try
    {
        {
            if (id != customerDto.Id)
            {
                return BadRequest("ID mismatch.");
            }
            await _customerService.UpdateCustomerDto(customerDto);

            response.IsSuccess = true;
            response.StatusCode = HttpStatusCode.OK;
            response.Result = new { message = "Customer updated successfully." };
            return Ok(response);
        }
        catch (Exception ex)
        {
            response.StatusCode = HttpStatusCode.InternalServerError;
            response.IsSuccess = false;
            response.ErrorMessages.Add($"Error: {ex.Message}");
            return StatusCode(500, response);
        }
    }
}

// Eliminar un asiento
[Authorize(Roles = "Admin")]
[HttpDelete("{id}")]
[ProducesResponseType(StatusCodes.Status204NoContent)]
[ProducesResponseType(StatusCodes.Status404NotFound)]
[ProducesResponseType(StatusCodes.Status500InternalServerError)]
public async Task<ActionResult> DeleteCustomerAsync(int id)
{
    var response = new RespuestaAPI();
    try
    {
        {
            await _customerService.DeleteCustomerAsync(id);
            return NoContent();
        }
        catch (KeyNotFoundException ex)
        {
            response.StatusCode = HttpStatusCode.NotFound;
            response.IsSuccess = false;
            response.ErrorMessages.Add($"Error: {ex.Message}");
            return NotFound(response);
        }
        catch (Exception ex)
        {
            response.StatusCode = HttpStatusCode.InternalServerError;
            response.IsSuccess = false;
            response.ErrorMessages.Add($"Error: {ex.Message}");

```

```

        return StatusCode(500, response);
    }
}

[Authorize(Roles = "Admin")]
[HttpGet("customer")]
[ProducesResponseType(StatusCodes.Status200OK)]
[ProducesResponseType(StatusCodes.Status403Forbidden)]
[ProducesResponseType(StatusCodes.Status500InternalServerError)]
public async Task<IActionResult> GetCustomerList()
{
    var response = new RespuestaAPI();

    try
    {
        var customers = await _customerService.ListCustomer();

        response.Result = customers;
        response.IsSuccess = true;
        response.StatusCode = (HttpStatusCode)(int)HttpStatusCode.OK;

        return Ok(response);
    }
    catch (Exception ex)
    {
        response.IsSuccess = false;
        response.StatusCode = (HttpStatusCode)(int)HttpStatusCode.InternalServerError;
        response.ErrorMessages.Add($"Error: {ex.Message}");

        return StatusCode(500, response);
    }

    // Redundante, pero garantiza que todas las rutas retornan algo
    // No debería alcanzarse nunca
    return StatusCode(500, new RespuestaAPI
    {
        IsSuccess = false,
        StatusCode = (HttpStatusCode)(int)HttpStatusCode.InternalServerError,
        ErrorMessages = new List<string> { "Ocurrió un error inesperado." }
    });
}
}
}

```

Análisis: ``html

Vulnerabilidades de Seguridad y Calidad del Código

1. Inyección de Excepciones sin Sanitización en Mensajes de Error

Tipo: Revelación de información sensible a través de excepciones.

Línea Aproximada: En todos los bloques catch, por ejemplo, dentro del método CreateCustomerAsync:

```
response.ErrorMessages.Add($"Error: {ex.Message}");
```

Descripción: Incluir el mensaje completo de la excepción directamente en la respuesta API puede revelar información sensible sobre la infraestructura interna, rutas de archivos, nombres de bases de datos, etc. Esto puede ser explotado por atacantes para obtener información valiosa sobre el sistema.

Mitigación: No retornar directamente el mensaje de la excepción al cliente. En su lugar, registrar la excepción completa en un sistema de logging (ej., Serilog, NLog) para análisis interno y retornar un mensaje de error genérico y seguro al cliente, como "Ocurrió un error inesperado. Consulte los registros para más detalles."

Métricas de Calidad: Mejora la seguridad (prevención de revelación de información) y la mantenibilidad (al facilitar el diagnóstico interno sin exponer detalles sensibles).

2. Manejo Inconsistente de Errores en GetCustomerByIdAsync

Tipo: Posible excepción no controlada.

Línea Aproximada:

```
var customer = await _customerService.GetCustomerIdAsync(id);
if (customer == null)
{
    return BadRequest("Invalid customer data.");
}
```

Descripción: Si `_customerService.GetCustomerIdAsync(id)` lanza una excepción *antes* de que se pueda verificar si `customer` es nulo, esta excepción no será capturada dentro del bloque `try...catch` actual. Esto podría resultar en una caída inesperada de la aplicación.

Mitigación: Asegurarse de que *todo* el código dentro de la acción esté protegido por el bloque `try...catch`:

```
try
{
    var customer = await _customerService.GetCustomerIdAsync(id);
    if (customer == null)
    {
        return NotFound("Customer not found."); // Mejor usar NotFound
    }

    response.Result = customer;
    response.IsSuccess = true;
    response.StatusCode = HttpStatusCode.OK;
    return Ok(response);
}
catch (Exception ex)
{
    response.StatusCode = HttpStatusCode.InternalServerError;
    response.IsSuccess = false;
    response.ErrorMessages.Add("An unexpected error occurred."); // Mensaje seguro
    // Loggear la excepción completa (ex) aquí
    return StatusCode(500, response);
}
```

Métricas de Calidad: Mejora la robustez del código, la capacidad de manejo de errores y la confiabilidad.

3. Validación Insuficiente en `UpdateCustomerAsync`

Tipo: Riesgo de Datos Inconsistentes.

Línea Aproximada:

```
if (id != customerDto.Id)
{
    return BadRequest("ID mismatch.");
}
await _customerService.UpdateCustomerDto(customerDto);
```

Descripción: Aunque se verifica la coincidencia de IDs, no hay validación adicional sobre el contenido del `customerDto`. Un atacante podría enviar un `customerDto` con datos inválidos o maliciosos.

Mitigación: Implementar validación robusta del `customerDto` utilizando `Data Annotations`, `FluentValidation` u otras bibliotecas de validación. Validar tipos de datos, rangos, longitudes, formatos, y cualquier otra restricción relevante.

Métricas de Calidad: Mejora la integridad de los datos, la robustez del código y reduce el riesgo de errores y vulnerabilidades.

4. Retorno Redundante al Final de GetCustomerList

Tipo: Código innecesario.

Línea Aproximada:

```
return StatusCode(500, new RespuestaAPI
{
    IsSuccess = false,
    StatusCode = (HttpStatusCode)(int)HttpStatusCode.InternalServerError,
    ErrorMessages = new List { "Ocurrió un error inesperado." }
});
```

Descripción: Este bloque de código es inalcanzable ya que todas las posibles rutas de ejecución dentro del método retornan un IActionResult antes de llegar a este punto.

Mitigación: Eliminar el código redundante. Esto simplifica el código y mejora la legibilidad.

Métricas de Calidad: Mejora la legibilidad, mantenibilidad y reduce la complejidad innecesaria.

Solución Propuesta

Refactorización General

El siguiente código propone una refactorización que aborda las vulnerabilidades y mejoras de calidad del código identificadas.

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using System;
using System.Collections.Generic;
using System.Net;
using System.Threading.Tasks;
using FermaOrders.Application.Dto.Components.Customer;
using FermaOrders.Application.Interface.Components;
using FermaOrders.API.Controllers.Response;
using Microsoft.Extensions.Logging; // Añadido el using

namespace FermaOrders.API.Controllers.Application
{
    [Route("api/[controller]")]
    [ApiController]
    public class CustomerController : ControllerBase
    {
        private readonly ICustomerService _customerService;
        private readonly ILogger<CustomerController> _logger; // Inyección del logger

        public CustomerController(ICustomerService customerService,
            ILogger<CustomerController> logger)
        {
            _customerService = customerService;
            _logger = logger;
        }

        [Authorize(Roles = "Admin")]
        [HttpPost]
        [ProducesResponseType(StatusCodes.Status200OK)]
        [ProducesResponseType(StatusCodes.Status400BadRequest)]
        [ProducesResponseType(StatusCodes.Status500InternalServerError)]
        public async Task<IActionResult> CreateCustomerAsync([FromBody] CustomerCreatedDto customerDto)
        {
            var response = new RespuestaAPI();

            if (customerDto == null)
            {
                response.IsSuccess = false;
                response.StatusCode = (int)HttpStatusCode.BadRequest;
                response.ErrorMessages = new List { "El cliente no puede ser null." };
                return response;
            }

            response.IsSuccess = true;
            response.StatusCode = (int)HttpStatusCode.Created;
            response.ErrorMessages = new List { "Cliente creado exitosamente." };
            return response;
        }
    }
}
```

```

        return BadRequest("Invalid customer data.");
    }
    try
    {
        var customer = await _customerService.CreateCustomerAsync(customerDto);
        response.Result = customer;
        response.IsSuccess = true;
        response.StatusCode = HttpStatusCode.OK;
        return Ok(response);
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, "Error creating customer."); // Log de la excepción
        response.StatusCode = HttpStatusCode.InternalServerError;
        response.IsSuccess = false;
        response.ErrorMessages.Add("An unexpected error occurred."); // Mensaje
        return StatusCode(500, response);
    }
}

[Authorize(Roles = "Admin")]
[HttpGet("{id}")]
[ProducesResponseType(StatusCodes.Status200OK)]
[ProducesResponseType(StatusCodes.Status404NotFound)]
[ProducesResponseType(StatusCodes.Status500InternalServerError)]
public async Task<IActionResult> GetCustomerByIdAsync(int id)
{
    var response = new RespuestaAPI();

    try
    {
        var customer = await _customerService.GetCustomerIdAsync(id);
        if (customer == null)
        {
            return NotFound("Customer not found.");
        }

        response.Result = customer;
        response.IsSuccess = true;
        response.StatusCode = HttpStatusCode.OK;
        return Ok(response);
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, "Error retrieving customer with ID {CustomerId}.", id);
        response.StatusCode = HttpStatusCode.InternalServerError;
        response.IsSuccess = false;
        response.ErrorMessages.Add("An unexpected error occurred.");
        return StatusCode(500, response);
    }
}

[Authorize(Roles = "Admin")]
[HttpPut("{id}")]
[ProducesResponseType(StatusCodes.Status200OK)]
[ProducesResponseType(StatusCodes.Status400BadRequest)]
[ProducesResponseType(StatusCodes.Status404NotFound)]
[ProducesResponseType(StatusCodes.Status500InternalServerError)]
public async Task<IActionResult> UpdateCustomerAsync(int id, [FromBody]
CustomerUpdatedDto customerDto)
{
    var response = new RespuestaAPI();

    if (customerDto == null)
    {
        return BadRequest("Invalid customer data.");
    }

    if (id != customerDto.Id)
    {
        return BadRequest("ID mismatch.");
    }

    // TODO: Implementar validación del customerDto usando Data Annotations o

```

FluentValidation.

```
    try
    {
        await _customerService.UpdateCustomerDto(customerDto);

        response.IsSuccess = true;
        response.StatusCode = HttpStatusCode.OK;
        response.Result = new { message = "Customer updated successfully." };
        return Ok(response);
    }
    catch (KeyNotFoundException ex)
    {
        _logger.LogError(ex, "Customer with ID {CustomerId} not found.", id);
        return NotFound(); // No es necesario usar la response si ya retornas
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, "Error updating customer with ID {CustomerId}.", id);
        response.StatusCode = HttpStatusCode.InternalServerError;
        response.IsSuccess = false;
        response.ErrorMessages.Add("An unexpected error occurred.");
        return StatusCode(500, response);
    }
}

[Authorize(Roles = "Admin")]
[HttpDelete("{id}")]
[ProducesResponseType(StatusCodes.Status204NoContent)]
[ProducesResponseType(StatusCodes.Status404NotFound)]
[ProducesResponseType(StatusCodes.Status500InternalServerError)]
public async Task<IActionResult> DeleteCustomerAsync(int id)
{
    try
    {
        await _customerService.DeleteCustomerAsync(id);
        return NoContent();
    }
    catch (KeyNotFoundException ex)
    {
        _logger.LogError(ex, "Customer with ID {CustomerId} not found.", id);
        return NotFound();
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, "Error deleting customer with ID {CustomerId}.", id);
        return StatusCode(500, new RespuestaAPI { IsSuccess = false, StatusCode =
HttpStatusCode.InternalServerError, ErrorMessages = new List<string> { "An unexpected error
occurred." } });
    }
}

[Authorize(Roles = "Admin")]
[HttpGet("customer")]
[ProducesResponseType(StatusCodes.Status200OK)]
[ProducesResponseType(StatusCodes.Status403Forbidden)]
[ProducesResponseType(StatusCodes.Status500InternalServerError)]
public async Task<IActionResult> GetCustomerList()
{
    var response = new RespuestaAPI();

    try
    {
        var customers = await _customerService.ListCustomer();

        response.Result = customers;
        response.IsSuccess = true;
        response.StatusCode = HttpStatusCode.OK;

        return Ok(response);
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, "Error retrieving customer list.");
        response.IsSuccess = false;
        response.StatusCode = HttpStatusCode.InternalServerError;
    }
}
```

```
        response.ErrorMessage.Add("An unexpected error occurred.");
        return StatusCode(500, response);
    }
    // Código redundante eliminado.
}
}
```

Cambios Clave:

- Inyección de ILogger para registro robusto de excepciones.
- Mensajes de error genéricos en las respuestas de la API.
- Registro de la excepción completa utilizando el logger para análisis interno.
- Eliminación del retorno redundante en GetCustomerList.
- Manejo de excepciones extendido para cubrir todo el código dentro de las acciones.
- Manejo de errores más conciso y eficiente (ej., usar NotFound() directamente).

...